

Безопасная разработка (Secure Coding) Детали и ответы на вопросы

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Виктор Дзицкий

Team Lead .NET Developer

- Руководитель группы разработки в SolarLab
- В промышленной разработке на C# больше 13 лет
- Сертификация MCSD
- Преподаватель курса "C# Basic", "C# Professional", "Asp.Net" в ОТУС

 @dzitskiy_dev, @dzitskiy_net

 dzitskiy@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **OTUS C#-
Advanced-2025-12**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Карта курса



СТАРТ

Управление памятью
на уровне ядра

Асинхронность и
многопоточность
для экспертов

Внутренние
механизмы CLR

Профилирование и
экстремальная
оптимизация

Современные
парадигмы и
инструменты эксперта

Проектный
модуль

ФИНИШ



Цели вебинара

К концу занятия вы сможете

1. Применять defensive programming
2. Защищать API от XSS, DoS и malicious input
3. Валидировать данные от недоверенных источников
4. Анализировать уязвимости зависимостей
5. Строить production-ready ASP.NET Core сервисы



Цели вебинара

К концу занятия вы сможете

Техники защитного программирования

Защита от атак на отказ в обслуживании

Валидацию данных от
недоверенных источников

Смысл

Для чего вам это уметь

Сделать приложение надежным
и максимально защищенным



Маршрут вебинара

Техники защитного программирования

Защиту от атак на отказ в обслуживании

Валидация данных от недоверенных источников

Анализ зависимостей проекта

Рефлексия

Безопасная разработка в ASP.NET Core

Безопасная разработка

О чем сегодня поговорим:

- Secure Coding: защита от атак и уязвимостей
- Defensive Programming
- XSS
- DDoS
- Sanitization
- Validation
- Dependency Security
- Практический подход для production backend



Secure Coding в ASP.NET Core

- **Defensive Programming**

Defensive programming предполагает, что любой внешний ввод потенциально опасен. Сервис обязан проверять размеры payload, допустимые символы, таймауты и ограничения ресурсов.

- **XSS**

Cross-Site Scripting позволяет злоумышленнику выполнить JavaScript в браузере пользователя. Stored XSS особенно опасен, так как payload хранится на сервере.

- **Sanitization**

Sanitization удаляет опасные HTML-конструкции и dangerous attributes. Важно понимать, что sanitization не заменяет validation.

- **DDoS**

DDoS-атаки часто возникают не из-за большого количества запросов, а из-за дорогих операций и отсутствия ограничений ресурсов.

- **Dependency Security**

Даже безопасный код может быть уязвим через сторонние зависимости и transitive packages.



Never Trust the Client

Источники данных:

- Недоверенный источник: браузер, mobile app, webhook, partner API
- Любые входные данные должны валидироваться
- Даже internal API нельзя считать безопасными
- Zero Trust как архитектурный подход



Типовые уязвимости ASP.NET

Виды уязвимостей:

- SQL Injection
- XSS (Cross-Site Scripting)
- SSRF
- Mass Assignment
- Broken Access Control
- DoS / Resource Exhaustion
- Insecure Deserialization



Валидация входящих данных

Что следует валидировать:

- Проверка длины строк
- Whitelist validation
- Проверка enum и диапазонов
- Ограничение JSON depth
- Request size limits
- Fail Fast подход



Whitelist vs Blacklist

Whitelist vs Blacklist

Работа со списками:

- **Blacklist:** запрещаем известные плохие символы
- **Whitelist:** разрешаем только допустимые
- Whitelist безопаснее и предсказуемее
- Regex + строгие ограничения форматов



Пример whitelist validation

Примеры:

- Разрешаем только латиницу, цифры и underscore
- Regex: `^[a-zA-Z0-9_]{3,20}$`
- Подходит для usernames, tenant IDs, aliases



XSS: Cross-Site Scripting

XSS: Cross-Site Scripting

Чего следует опасаться:

- Stored XSS
- Reflected XSS
- DOM-based XSS
- Исполнение JavaScript в браузере жертвы
- Кража cookies, session hijacking, phishing



XSS: Cross-Site Scripting

Примеры:

- Stored XSS
- Reflected XSS
- DOM-based XSS
- Исполнение JavaScript в браузере жертвы
- Кража cookies, session hijacking, phishing



Защита от XSS

Как обезопасить приложение:

- HTML Encoding
- Content Security Policy (CSP)
- Запрет inline scripts
- Canitization HTML
- Экранирование пользовательского ввода
- Никогда не рендерить raw HTML без фильтрации



**Очистка пользовательского
ввода от потенциально
опасного содержимого**

Sanitization

Sanitization (санитизация) — это процесс очистки пользовательского ввода от потенциально опасного содержимого перед его отображением, сохранением или обработкой.

Главная цель - не допустить выполнение вредоносного кода, скриптов или опасных конструкций:

- Sanitization удаляет опасные конструкции
- Удаление script-tag, onclick, onerror
- Фильтрация dangerous attributes
- DOMPurify / Ganss.XSS для .NET
- Sanitization != Validation



Экранирование и Encoding

- HTML Encoding
- URL Encoding
- JavaScript Encoding
- SQL parameters вместо string concatenation
- Контекстно-зависимая защита

Защита от запуску скриптов

- Content-Security-Policy
- X-Content-Type-Options
- Запрет inline JS
- HttpOnly cookies
- SameSite cookies
- iframe sandbox



Защита от атак на отказ в обслуживании (Denial of Service)

DDoS и Resource Exhaustion

- Большие JSON payloads
- Slow requests
- Отсутствие pagination
- Тяжелые LINQ queries
- Блокировка thread pool
- Memory pressure и GC storms



Rate Limiting

- ASP.NET Core RateLimiter middleware
- Fixed Window
- Sliding Window
- Token Bucket
- Защита от brute force и flood



Timeouts и CancellationToken

- Всегда использовать CancellationToken
- HttpClient timeout
- DB timeout
- Fail fast вместо hanging requests



Защита от DDoS-атак: Управление ресурсами

Ограничение размера запросов:

- **В ASP.NET Core:** `[RequestSizeLimit(100_000)]` (атрибут), `services.Configure<KestrelServerOptions>(...)` или `services.Configure<IISServerOptions>(...)` на уровне приложения.
- **Для IFormFile:** `[MaxFileSize(5 * 1024 * 1024)]`.

Таймауты:

- **HTTP-запросы:** `HttpClient.Timeout`. Используйте `CancellationTokenSource` с таймаутом.
- **База данных:** `CommandTimeout` в `SqlCommand` или настройках EF Core.
- **Асинхронные операции:** Всегда передавайте `CancellationToken` в асинхронные методы, поддерживающие его.
- **Общий таймаут приложения:** В ASP.NET Core: `UseTimeoutMiddleware` или настройки хостинга.



Защита от DoS-атак: Управление ресурсами

Ограничение скорости запросов (Rate Limiting):

- **ASP.NET Core 7+:** Встроенный RateLimiter.
Пример SlidingWindowLimiter.
Можно применять глобально или к конкретным endpoint'ам.
- **Пакеты:** AspNetCoreRateLimit.
- **Зачем:** Защита от брутфорса (логин, API-ключи) и от простых flood-атак.

Осторожно с «тяжелыми» операциями:

- Синхронный I/O в асинхронном контексте (.Result, .Wait()) — путь к истощению пула потоков.
- **Регулярные выражения:** Сложные паттерны с неограниченными квантификаторами (+, *) уязвимы к ReDoS. Используйте RegexOptions.NonBacktracking (.NET 7+) или лимитируйте время выполнения.
- **Десериализация:** Большие JSON/XML в объекты могут съесть память. Контролируйте глубину, количество элементов.



Безопасность зависимостей

Безопасность зависимостей

- `dotnet list package --vulnerable`
- OWASP Dependency Check
- Dependabot
- Snyk
- Проблема malicious packages



Культура логирования

Безопасное логирование

- Не логировать пароли и токены
- PII masking
- Structured logging
- Correlation IDs
- Security audit trails



Анализ зависимости проекта на предмет известных уязвимостей

Безопасность зависимостей (Software Composition Analysis - SCA)

Проблема: Ваше приложение безопасно, но библиотека AwesomeParser v1.0.0 содержит RCE.

Инструменты:

- *dotnet list package --vulnerable* — встроенная команда CLI. Быстрая проверка.
- *dotnet retire* — устарел, но концепция актуальна.
- **NuGet Audit (.NET 8+):** Автоматическая проверка при восстановлении пакетов.
- **Сторонние решения:** OWASP Dependency-Check, Snyk, GitHub Dependabot, WhiteSource Bolt.



Безопасность зависимостей (Software Composition Analysis - SCA)

Рекомендации:

- Интегрируйте проверку в CI/CD пайплайн.

Не выпускайте сборку с критическими уязвимостями.

- Регулярно обновляйте пакеты (но с тестированием!).
- Используйте *Directory.Packages.props* (Central Package Management) для единого управления версиями в больших решениях.



Практический пример

Вопросы для обсуждения

- Почему internal network нельзя считать безопасной?
- Что опаснее: 1 huge request или 1000 small requests?
- Почему GUID не является защитой?
- Можно ли доверять JWT после проверки подписи?

**Подведём
итоги занятия**

Итоги

- **Безопасность** — часть архитектуры
- **Secure Coding** — обязанность разработчика
- **Главная угроза** — обычные инженерные ошибки
- *Безопасность нужно проектировать заранее!*



Список материалов для изучения

- [Рекомендации по безопасному написанию кода](#)
- [Правила безопасности](#)
- [.NET Security – это просто](#)
- [DotNet Security Cheat Sheet](#)
- [Secure Coding Practices in C#: Protecting Your Application](#)
- [Secure Coding Practices for Microsoft .NET Applications](#)
- [OWASP Cheat Sheet Series](#)
- [OWASP Top 10:2025](#)
- [Understanding the OWASP Top 10 across the Years: Common Weaknesses and Notorious Attacks](#)
- [OWASP Smart Contract Top 10](#)
- [Какие уязвимости входят в OWASP Top 10](#)
- [C# Security: Best Practices for Secure Coding](#)
- [Microsoft Security Development Lifecycle \(SDL\)](#)





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

